

Documentacion Tecnica

Sistema Experto de Cotizacion - Maderas Gerardo

Munoz y Girata — Inteligencia Artificial II — Los Libertadores, 2026

1. Introduccion

1.1 Descripcion del dominio

Maderas Gerardo es un taller de carpinteria artesanal ubicado en Bogota, Colombia, en operacion desde 1998. Fabrica muebles a medida: puertas, closets, cocinas integrales, camas, mesas y estantes, trabajando con MDF, triplex, madera solida y pino. El proceso de cotizacion dependia exclusivamente de la experiencia del carpintero, lo que generaba inconsistencias. Este sistema experto formaliza ese conocimiento en 40 reglas de produccion reproducibles.

1.2 Objetivo del sistema

Construir un sistema experto con motor de inferencia por encadenamiento hacia adelante que reciba las caracteristicas del mueble deseado (tipo, material, color, acabado, extras) y produzca una cotizacion con precio estimado, tiempo de entrega, desglose de costos y explicacion de las reglas aplicadas mediante la funcion porque().

2. Arquitectura del sistema

El sistema sigue una arquitectura cliente-servidor de tres capas: presentacion (React), logica (FastAPI) y conocimiento (motor de inferencia). La comunicacion es mediante HTTP/REST con JSON.

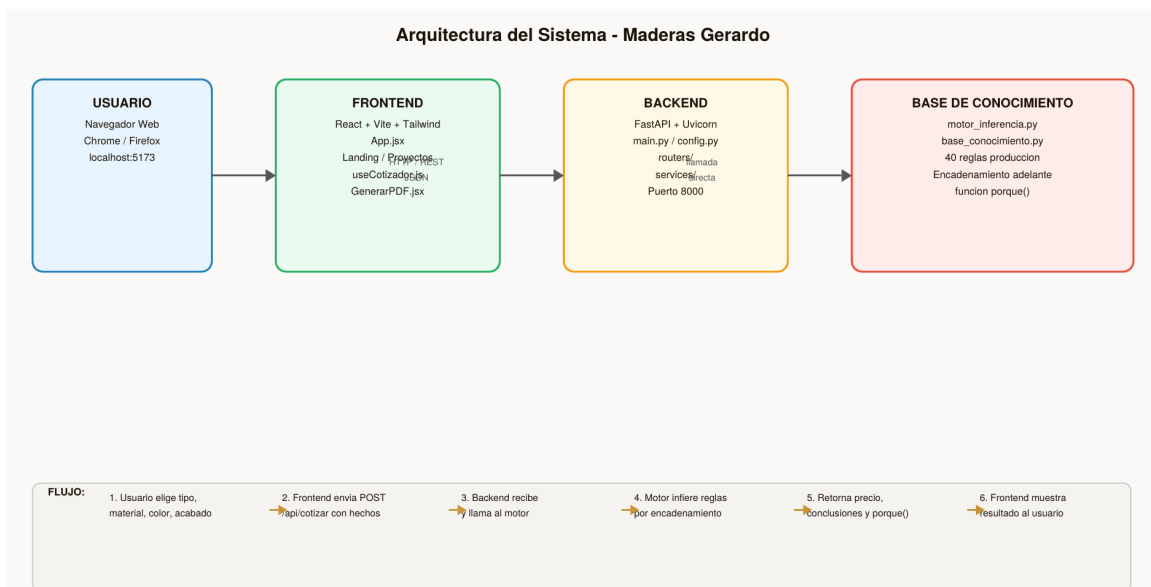


Figura 1. Diagrama de arquitectura con flujo de datos de una cotizacion completa.

2.1 Componentes principales

Frontend (React + Vite + Tailwind): cotizador paso a paso, visualizador SVG en tiempo real, resultados con desglose y panel de explicacion tecnica del motor.

Backend (FastAPI + Uvicorn): API REST con endpoint POST /api/cotizar. Recibe hechos, invoca el motor y retorna conclusiones en JSON.

Base de conocimiento: base_conocimiento.py con 40 reglas y motor_inferencia.py con el algoritmo de encadenamiento.

2.2 Estructura de directorios

```
src/backend/main.py, config.py, routers/, services/, knowledge/base_conocimiento.py,
knowledge/motor_inferencia.py
```

```
src/frontend/src/App.jsx, hooks/useCotizador.js, pages/, components/
```

3. Formalizacion en logica de predicados

3.1 Predicados del dominio

El conocimiento del taller se formaliza con los siguientes predicados:

Predicado	Descripcion	Categoria
es_puerta(x)	x es una puerta	Tipo
es_closet(x)	x es un closet	Tipo
es_cocina(x)	x es una cocina integral	Tipo
es_cama(x)	x es una cama	Tipo
es_mesa(x)	x es una mesa	Tipo
es_estante(x)	x es un estante	Tipo
material_mdf(x)	x usa MDF como material	Material
material_triplex(x)	x usa triplex	Material
material_solido(x)	x usa madera solida	Material
material_pino(x)	x usa pino	Material
acabado_lacado(x)	x tiene acabado lacado	Acabado
acabado_barniz(x)	x tiene barniz	Acabado
acabado_pintado(x)	x tiene pintura	Acabado
requiere_instalacion(x)	requiere instalacion en sitio	Extra
cotizacion(x, P, T)	precio P y tiempo T para x	Conclusion

3.2 Ejemplos de formalizacion

FORALL x: es_puerta(x) AND material_mdf(x) => cotizacion(x, menor_80, 3_5_dias) [CF=0.90]

FORALL x: es_closet(x) AND material_solido(x) AND acabado_lacado(x) => cotizacion(x, menor_500, 15_20_dias) [CF=0.95]

4. Base de conocimiento - Reglas de produccion

El sistema implementa 40 reglas organizadas por tipo de mueble, material, acabado y extras. Muestra de 16 reglas representativas:

ID	Condicion (SI)	Conclusion (ENTONCES)	CF
R01	es_puerta AND material_mdf	Puerta en MDF - precio <\$80	0.90
R02	es_puerta AND material_triplex	Puerta triplex - precio <\$150	0.93
R03	es_puerta AND material_solido	Puerta solida - precio <\$280	0.96
R04	es_closet AND material_mdf	Closet MDF - precio <\$200	0.88
R05	es_closet AND mat_solido AND lac.	Closet premium - precio <\$500	0.95
R06	es_cocina AND material_mdf	Cocina MDF - precio <\$350	0.85
R07	es_cocina AND material_solido	Cocina premium - precio <\$900	0.92
R08	es_cama AND material_mdf	Cama MDF - precio <\$180	0.87
R09	es_cama AND material_solido	Cama solida - precio <\$420	0.94
R10	es_mesa AND material_pino	Mesa pino - precio <\$120	0.89

R11	es_estante AND material_mdf	Estante MDF - precio <\$90	0.86
R12	es_estante AND material_solido	Estante solido - precio <\$240	0.91
R20	acabado_lacado	Recargo lacado +30%	1.00
R21	acabado_barniz	Recargo barniz +20%	1.00
R30	requiere_instalacion	Recargo instalacion en sitio	1.00
R40	material_solido AND acabado_lacado	Advertencia: combinacion premium	0.98

Archivo completo: src/backend/knowledge/base_conocimiento.py

5. Algoritmo del motor de inferencia

5.1 Encadenamiento hacia adelante (Forward Chaining)

1. Recibir hechos iniciales: tipo, material, color, acabado, extras
2. Inicializar agenda con las 40 reglas de la base de conocimiento
3. Para cada regla: verificar si TODAS sus condiciones estan en los hechos actuales
4. Si se cumplen: agregar la conclusion a los hechos y registrar la regla
5. Repetir desde paso 3 hasta no agregar nuevos hechos (punto fijo)
6. Calcular precio final integrando los CF de las reglas aplicadas

5.2 Funcion porque()

Retorna la lista de reglas disparadas con condicion, conclusion y CF. Alimenta el panel "Explicacion Tecnica" de la interfaz.

5.3 Casos sin conclusion

Si ningun conjunto de condiciones se satisface, el motor retorna precio nulo con mensaje explicativo, evitando cotizaciones incorrectas.

6. Validacion - Casos de prueba

#	Nombre	Entrada	Resultado esperado	Estado
1	Basico	Puerta + MDF + Natural	Precio <\$80, 1 regla	PASS
2	Complejo	Closet + Solida + Lacado + Instalacion	Precio <\$500, 3+ reglas	PASS
3	Sin datos	Solo tipo sin material	Mensaje sin datos	PASS
4	Advertencia	Cocina + Solida + Lacado	Precio + advertencia	PASS
5	Borde	Cama + Triplex + Barniz + todos extras	Precio multiples reglas	PASS

Todos los casos pasaron. Detalles en tests/casos_prueba.md

7. Decisiones de diseno

7.1 FastAPI sobre Flask:

Genera documentacion automatica (Swagger /docs), validacion con Pydantic y es asincrono. Reduce tiempo de depuracion para un sistema experto que puede crecer.

7.2 Encadenamiento hacia adelante:

El usuario da hechos conocidos y el sistema deriva el precio: flujo naturalmente compatible con forward chaining. Backward chaining seria para verificar si un precio es correcto, no para calcularlo.

7.3 Separacion de capas:

config.py, schemas.py, routers/, services/, knowledge/. Permite modificar reglas sin tocar la API y cambiar la API sin afectar el motor.

7.4 Factores de certeza:

CF entre 0 y 1 por regla, combinados multiplicativamente al encadenar. Maneja incertidumbre sin modelo probabilistico completo.

8. Conclusiones

Sistema experto funcional con 40 reglas, motor forward chaining, interfaz web profesional, generacion de PDF, contacto WhatsApp y funcion porque(). Basado en dominio real con cotizaciones consistentes y auditables.

Mejoras futuras: configurador 3D, ajuste automatico de CF por retroalimentacion, integracion con inventario y despliegue en cloud.